Fine-tune a language model with Microsoft Foundry

Thursday, March 12, 2026 3:20 PM

1. **Introduction:**
    a. Imagine you're a developer working for a travel agency. When customers use your chat application to get help with their travel-related questions, you want the responses to be in a specific format and style. Your company has a specific tone of voice that resonates with your client base. The marketing department finds it important that the chat application is aligned with your company's tone of voice too.
    b. There are various strategies to optimize the model's behavior and the performance of your chat application. One strategy is to fine-tune a language model, which you can then integrate with your chat application. The benefit of fine-tuning over training your own language model, is that you need less time, compute resources, and data to customize the model to your needs.
2. **Understand when to fine-tune a language model:**
    a. Types of fine tuning:
        i. Supervised fine-tuning for training with labeled examples
        ii. Function calling fine-tuning for structured output and API integration
        iii. Vision fine-tuning for image understanding tasks
        iv. Reinforcement fine-tuning for reward-based learning
        v. Direct preference optimization for alignment using preference pairs
    b. How the model needs to act mostly relates to the style, format, and tone of the responses generated by a model. When you want your model to adhere to a specific style and format when responding, you can instruct the model to do so through prompt engineering too. Sometimes however, prompt engineering might not lead to consistent results. It can still happen that a model ignores your instructions and behaves differently.
    c. Within prompt engineering, a technique used to "force" the model to generate output in a specific format, is to provide the model with various examples of what the desired output might look like, also known as one-shot (one example) or few-shot (few examples). Still, it can happen that your model doesn't always generate the output in the style and format you specified.
    d. To maximize the consistency of the model's behavior, you can fine-tune a base model with your own training data.
    e. Microsoft Foundry supports multiple fine-tuning techniques, each designed for different use cases and model capabilities:
        i. Supervised fine-tuning:
            1. Supervised fine-tuning is the most basic and common approach where you train a base model on labeled example data. You provide the model with example conversations that demonstrate the desired behavior, including system messages, user prompts, and assistant responses. This technique is ideal for teaching the model specific formats, styles, tones, or domain-specific behaviors.
            2. Supervised fine-tuning is supported for models like GPT-4, GPT-4o, GPT-3.5-Turbo, and many other foundation models in the model catalog. This is the recommended starting point for most fine-tuning scenarios.
        ii. Reinforcement fine-tuning:
            1. Reinforcement fine-tuning (RFT) is an advanced technique that improves reasoning models by training them through a reward-based process, rather than relying only on labeled data. Instead of providing example responses, you provide prompts and a grader that scores the quality of the model's outputs. The model learns to generate better responses by maximizing the reward signal.
            2. RFT is especially useful for:
                a. Complex reasoning and problem-solving tasks
                b. Scenarios where labeled examples are limited
                c. Cases where you want the model to develop sophisticated reasoning strategies
            3. RFT is supported for advanced reasoning models like o4-mini and gpt-5. When using RFT, you need to define a grader (such as text comparison, model-based, or custom code graders) that evaluates the model's outputs during training.
                a. Complex reasoning and problem-solving tasks
                b. Scenarios where labeled examples are limited
                c. Cases where you want the model to develop sophisticated reasoning strategies
            4. Direct preference optimization (DPO):
                a. Direct preference optimization (DPO) is an advanced alignment technique that adjusts model weights based on human preferences. Instead of providing single example responses, you provide pairs of responses - one preferred and one nonpreferred - for each prompt. The model learns to generate outputs more similar to the preferred examples.
                b. DPO is especially useful when:
                    i. There's no clear-cut correct answer
                    ii. Subjective elements like tone, style, or content preferences are important
                    iii. You have preference data from user logs, A/B tests, or manual annotations
                    iv. You want a computationally lighter alternative to reinforcement learning from human feedback (RLHF)
                c. DPO is supported for models like gpt-4o, gpt-4.1, and gpt-4.1-mini. You can use DPO with base models or with models already fine-tuned using supervised fine-tuning.
            5. Fine-tune for function calling:
                a. Function calling fine-tuning is an advanced technique that trains models to reliably call external functions or APIs with structured arguments. You provide training examples that demonstrate how the model should respond to user requests by calling specific functions with the correct parameters. This technique teaches the model when and how to use tools, improving its ability to generate structured output and integrate with external systems.

b. Function calling fine-tuning is especially useful for:
   i. Building agents that need to interact with APIs or databases
   ii. Ensuring consistent JSON schema adherence
   iii. Teaching domain-specific function usage patterns
   iv. Reducing errors in parameter extraction and function selection
c. Function calling fine-tuning is supported for models like GPT-4o and GPT-4o-mini. You can combine this with supervised fine-tuning to teach both conversational behavior and function calling capabilities.

6. Fine-tune for vision tasks
   a. Vision fine-tuning is an advanced technique that enhances models' ability to understand and reason about images. You provide training examples that pair images with text prompts and expected responses, teaching the model to recognize specific visual patterns, objects, or concepts relevant to your domain. This technique is ideal for specialized computer vision applications where general-purpose vision models need domain-specific understanding.
   b. Vision fine-tuning is especially useful for:
      i. Medical imaging analysis with specialized terminology
      ii. Industrial quality control and defect detection
      iii. Document understanding with custom layouts or formats
      iv. Domain-specific image classification and captioning

3. Prepare your data to fine-tune a chat completion model

a. Understand the training data format
   i. When you fine-tune a language model for chat completion, the data you use to fine-tune a model is a collection of sample conversations. More specifically, the data should contain three components:
      1. The system message
      2. The user message
      3. The assistant's response
   ii. {"messages": [{"role": "**system**", "content": "You are an Xbox customer support agent whose primary goal is to help users with issues they are experiencing with their Xbox devices. You are friendly and concise. You only provide factual answers to queries, and do not provide answers that are not related to Xbox."}, {"role": "**user**", "content": "Is Xbox better than PlayStation?"}, {"role": "**assistant**", "content": "I apologize, but I cannot provide personal opinions. My primary job is to assist you with any issues related to your Xbox device. Do you have any Xbox-related issues that need addressing?"}]}
b. Prepare datasets from real data
   i. The dataset should show the model's ideal behavior. You can create this dataset based on the chat history of a chat application you have. A few things to keep in mind when you use real data is to:
      1. The dataset should show the model's ideal behavior. You can create this dataset based on the chat history of a chat application you have. A few things to keep in mind when you use real data is to:
      2. Not only focus on creating a large training dataset, but also ensure your dataset includes a diverse set of examples.
   ii. You can include multiple turns of a conversation on a single line in the dataset. If you want to fine-tune only on specific assistant messages, you can optionally use the weight key-value pair. When the weight is set to 0, the message is ignored, when you set to 1, the message is included for training.
   iii. {"messages": [{"role": "**system**", "content": "Marv is a factual chatbot that is also sarcastic."}, {"role": "**user**", "content": "What's the capital of France?"}, {"role": "**assistant**", "content": "Paris", "**weight**": 0}, {"role": "**user**", "content": "Can you be more sarcastic?"}, {"role": "**assistant**", "content": "Paris, as if everyone doesn't know that already.", "**weight**": 1}]}
c. Generate synthetic data for fine-tuning:
   i. Creating high-quality training datasets manually can be time-consuming and expensive. Microsoft Foundry provides synthetic data generation capabilities that can help you create training data from reference documents or API specifications.
   ii. Synthetic data generation is especially useful when:
      1. Real data is scarce or difficult to collect
      2. You need to preserve privacy while retaining useful structure
      3. You want to generate domain-specific data from existing documents or code
      4. You want to reduce the cost compared to manual data collection
   iii. Choose synthetic data generators. Microsoft Foundry offers two types of synthetic data generators:
      1. Simple Q&A Generator: Converts domain documents (PDF, Markdown, or plain text up to 20 MB) into fine-tuning question-answer pairs. You can configure the question type:
         a. Long answer: Generates questions that require analytical reasoning in the model response
         b. Short answer: Generates questions focused on factual brevity
      2. Tool use Generator: Creates multi-turn conversations with tool calls based on your API surface. Requires a valid OpenAPI 3.0.x or 3.1.x specification (JSON format, up to 20MB) that describes the APIs you want the model to learn to call as tools.
   iv. Generate synthetic data in the portal:
      1.
      2.

To generate synthetic data in Microsoft Foundry:

1. Navigate to **Data** > **Synthetic Data Generation** in the portal
2. Select **Generate data**
3. Choose a task type (Simple Q&A or Tool use)
4. Configure generation parameters if applicable
5. Upload your reference file

6. Specify the number of samples to generate (between 50 and 1,000)
7. Select the model to use for data generation
8. Optionally enable an 80/20 train-validation split
9. Submit the job

3. The generated dataset is automatically formatted as JSONL files compatible with Microsoft Foundry fine-tuning workflows. You can download the dataset for review or use it directly in fine-tuning.

v. Experiment with hyperparameters: When fine-tuning on synthetic data, you need to adjust learning rates and other parameters differently than with real data to avoid overfitting.

vi. Monitor performance: Regularly evaluate your fine-tuned model on real-world tasks to ensure it meets your requirements.

4. **Explore fine-tuning language models in Microsoft Foundry portal:**
   a. You can fine-tune base models on various tasks, like text classification, translation, or chat completion.
   b. When you want to use a fine-tuned model to generate responses in a chat application, you need to use a base model that can be fine-tuned on a chat completion task. The Microsoft Foundry model catalog allows you to filter based on fine-tuning tasks to decide which base model to select. You can, for example, select a GPT-4 or Llama-2-7b model to fine-tune on your own training data.
   c. You can filter the available models based on the task you want to fine-tune a model for. Per task, you have several options for foundation models to choose from. When deciding between foundation models for a task, you can examine the description of the model, and the referenced model card.
   d. Configure the fine-tuning job
      i. Select a base model.
      ii. Select your training data.
      iii. (Optional) Select your validation data.
      iv. Configure the advanced options.
   e. When you submit a model for fine-tuning, the model is further trained on your data. To configure the fine-tuning or training job, you can specify the following advanced options:
      i. batch_size:
         1. The batch size to use for training. The batch size is the number of training examples used to train a single forward and backward pass.
         2. In general, larger batch sizes tend to work better for larger datasets. The default value and the maximum value for this property are specific to a base model.
         3. A larger batch size means that model parameters are updated less frequently, but with lower variance.
      ii. learning_rate_multiplier:
         1. The learning rate multiplier to use for training.
         2. The fine-tuning learning rate is the original learning rate used for pretraining multiplied by this value.
         3. Larger learning rates tend to perform better with larger batch sizes.
         4. We recommend experimenting with values in the range 0.02 to 0.2 to see what produces the best results.
         5. A smaller learning rate can be useful to avoid overfitting.
      iii. n_epochs:
         1. The number of epochs to train the model for. An epoch refers to one full cycle through the training dataset.
      iv. Seed:
         1. The seed controls the reproducibility of the job. Passing in the same seed and job parameters should produce the same results, but can differ in rare cases. If a seed isn't specified, one is generated for you.
   f. After you submit the fine-tuning job, a job will be created to train your model. You can review the status of the job while it's running. After the job is completed, you can review the input parameters when you want to understand how the fine-tuned model was created.
   g. If you added a validation dataset, you can review the model's performance by exploring how it performed on your validation dataset.
5. **Exercise:** https://microsoftlearning.github.io/mslearn-ai-studio/Instructions/05-Finetune-model.html
6. **Model Assesment:**
   a.
   b.

1. How must data be formatted for fine-tuning?
- JSONL ✓ Correct
- YAML
- HTML

2. What does fine-tuning optimize in your model?
- What the model needs to know.
- How the model needs to act. ✓ Correct
- Which words aren't allowed.

3. Which advanced option refers to one full cycle through the training dataset?
- seed
- batch_size
- n_epochs ✓ Correct

   c.